Primera entrega del proyecto:

simulador juego BUS

Javier Felipe Contreras Díaz
202535936
contreras.javier@correounivalle.edu.co

Jhoan Esteban Penagos Lizcano
2517537
Jhoan.penagos@correounivalle.edu.co

Maria Camila Zuleta Peña
2540542
mariacamilazuletapena@gmail.com

Facultad de Ingeniería, Universidad del Valle

750015C: Fundamentos de Programación Orientada a Objetos

Victor Andres Bucheli Guerrero

Junio de 2026

1. Diagnostico y requerimientos:

Para la realización del proyecto, en primer lugar es necesario realizar un diagnostico del requerimiento, es decir evaluar qué es lo que se está pidiendo y consolidarlo en objetivos con resultados esperados.

En este caso el sistema solicitado consiste e un tablero (grid) donde diversos buses de colores están bloqueados por el trafico. Siendo así se identifican 2 clases: una clase **Tablero** y otra clase **Vehículo**. Adicional a esto, habrán pasajeros que abordarán los distintos vehículos, razón por la cual habrá otra clase **Pasajero.** El objetivo es liberar los vehículos y permitir que los pasajeros aborden los distintos medios de transporte según la correspondencia de colores. Con base en este escenario, se han estructurado formalmente las Historias de Usuario (HU) y la matriz de Requerimientos Funcionales (RF):

Id: RF	Historias de Usuario	Descripción Detallada	Resultado Esperado
RF1	HU-1: Registro y Carga de Niveles	Carga Dinámica de Niveles .txt: El sistema debe leer archivos planos de cada nivel para procesar las dimensiones del tablero, la flota de vehículos, su posición inicial y la cola de pasajeros.	El archivo Niveles.txt procesa correctamente y posiciona cada objeto en las coordenadas correspondientes del tablero al arrancar.
RF2	HU-2: Gestión de Escenario	Inicialización del Escenario (Tablero): Ubicar dinámicamente cada vehículo (Vehiculo*) en la matriz bidimensional zonaCoches respetando su posición inicial (X, Y), tamaño y orientación.	El mapa se despliega en consola mostrando las posiciones exactas de los objetos y los obstáculos.
RF3	HU-3: Mecánicas de Movimiento	Validación de Colisiones y Límites: Verificar antes de cada movimiento que las celdas de destino estén vacías (nullptr) y dentro de los rangos válidos de la matriz	Bloqueo del movimiento si el coche choca contra otro o contra un borde/pared.
RF4	HU-3: Mecánicas de Movimiento	Desplazamiento: El usuario selecciona un vehículo por sus coordenadas y lo mueve exclusivamente en la dirección de su orientación lineal asignada.	Modificación exitosa de las variables posX y posY del objeto y su representación gráfica en consola.
RF5	HU-4: Mecánicas de Color	Abordaje por color: Procesar la cola de peatones. Si el color del primer pasajero coincide con el del vehículo estacionado en la zonaRecogida y el vehículo tiene cupo libre, este aborda liberando el espacio en la fila.	El pasajero pasa de la fila general al vector sillas del vehículo.
RF6	HU-5: Lógica de victoria/derrota	Detección de Estado de Derrota: Evaluar si la zonaRecogida excede la capacidad máxima de estacionamiento (4 vehículos) o si se encuentra completamente bloqueada sin movimientos posibles.	Terminación del juego -Game Over- según si se logran o se pierden los objetivos.

2. Arquitectura y diagrama de clases:

Según el resultado funcional que se busca y siguiendo los lineamientos del proyecto Modelo-Vista-Controlador (MVC):

• Modelo: Administra el estado interno de la simulación, la carga de archivos planos, las reglas de colisión, la colección de vehículos y la lógica del turno (Clases Tablero, Vehículo, Pasajero y Modelo).

• Vista (View): Encargada del renderizado del tablero, vehículos, pasajeros y juego en general en la interfaz gráfica o consola.

• Controlador (Controller): Captura las entradas por teclado del usuario, valida que los comandos sean válidos y los transmite siguiendo la lógica principal del juego.

3. Código del diagrama de clases en planUML:

```
@startuml

class Controller{

-nivel: int
-vista: View&
-model: Model&

+Controller(Model& auxModelo, View& auxVista)
+iniciarJuego() : bool
+movimientosVehiculos() : bool
+mostrarMapa(): void
+pushPasajerosBus(): void
+verificarZonaRecogida(): void
+verificarVictoria() : bool
+verificarDerrota() : bool
}


class Model{
-Vehiculos: vector<Vehiculo>
-Pasajeros: vector<Pasajero>
-tablero: Tablero

+Model()
+cargarTxt(int nivel): void
+cargarVehiculosEnTablero(): void
+cargarPasajerosEnTablero(): void
+verificarColision(int posX, int posY): bool
+verificarDireccion(char direccion, int posX, int posY): bool
+verificarInputDireccion(): char
+verificarPosicionVehiculo(): tuple<int,int>
```

+verificarZonaRecogida(): void
+moverVehiculo(int posX, int posY, int tam,char direccion): bool
+pushPasajerosBus(int i): void
+getVerificarDerrota(): bool
+getTablero(): Tablero&
}

abstract class View{
+View():
+{abstract} mostrarMenu(): void
+{abstract} mostrarNivel(int nivel): void
+{abstract} mostrarMapa(Tablero& paradero): void
+{abstract} pantallaDerrota() : void
+{abstract} pantallaVictoria(): void
+virtual ~View():

}

abstract class ViewSFML{
+ViewSFML()
+mostrarMenu(): void override
+mostrarNivel(int nivel): void override
+mostrarMapa(Tablero& paradero): void override
+pantallaDerrota() : void override
+pantallaVictoria(): void override
+~ViewSFML(): override
}

abstract class ViewConsola{
+ViewConsola()
+mostrarMenu(): void override
+mostrarNivel(int nivel): void override
+mostrarMapa(Tablero& paradero): void override
+pantallaDerrota() : void override
+pantallaVictoria(): void override
+~ViewConsola(): override
}

class Vehiculo{

-tipo: char
-color: char
-orientacion: char
-tamano: int
-posX: int
-posY: int
-sillas: vector Pasajero*

+Vehiculo(char auxTipo,char auxColor,char auxOrientacion, int auxTamano, int auxPosX, int auxPosY)
+getColor(): char
+getTipo(): char
+getTamano(): int
+getOrientación(): char
+getPosX(): int
+getPosY(): int
+getSizeSillas(): int
+setPos(int x,int y): void
+pushPasajerosBus(Pasajero* peaton): void
}

class Pasajero{
-color: char
+Pasajero(char auxColor)
+getColor(): char
}

class Tablero{

-filas: int
-columnas: int
-zonaCoches: vector<vector<Vehiculo*>>
-filaPasajeros: vector<Pasajero*>
-zonaRecogida: vector<Vehiculo*>
-zonaVictoria: vector<Vehiculo*>
-verificarDerrota: bool
+Tablero()
+setZonaCoches(Vehiculo* auxVehiculo): void
+pushFilaPasajeros(Pasajero*): void
+verificarColision(int posX,int posY): bool
+verificarDireccion(char direccion, int posX, int posY): bool
+verificarDerrota(): bool
+moverVehiculo(int posX, int posY, int tam): bool
+pushPasajerosBus(Pasajero* peaton, int i): void
+verificarZonaRecogida():void
+pushZonaVictoria(Vehiculo* carro):void

+getFilaPasajeros(): vector<Pasajero*>&
+getZonarecogida(): vector<Vehiculo*>&
+getZonaCoches(): vector<Vector<Vehiculo*>>&
+getZonaVictoria(): vector<Vehiculo*>&
+getFilas(): int
+getColumnas(): int
+verificarPushPasajeros(): bool

```
}



Model *-- Vehiculo
Model *-- Pasajero
Model *-- Tablero

Tablero o-- Vehiculo
Tablero o-- Pasajero

Controller --> Model
Controller --> View

View <|-- ViewConsola
View <|-- ViewSFML
View ..> Tablero
@enduml
```